

Weightless neuro-symbolic GPS trajectory classification

Raul Barbosa^a, Douglas O. Cardoso^{a,b}, Diego Carvalho^{b,*}, Felipe M.G. França^a

^a Universidade Federal do Rio de Janeiro, PESC-COPPE, RJ, Brazil

^b Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, PPPRO, RJ, Brazil



ARTICLE INFO

Article history:

Received 13 July 2017

Revised 22 October 2017

Accepted 20 November 2017

Available online 21 February 2018

MSC:

00–01

99–00

Keywords:

Neuro-symbolic learning system

Trajectory classification

Binarization methods

WiSARD

ABSTRACT

This paper presents a framework for dealing with the problem of GPS trajectory classification in the context of the Rio de Janeiro's public transit system (with hundreds or more classes). Such framework combines the versatile WiSARD classifier with a set of rules defined *a priori*, resulting in a neuro-symbolic learning system with very interesting characteristics and cutting-edge performance. We also verified the influence of different binarization methods in order to adapt raw data to WiSARD, which feeds from binary data only. These ideas were tested against a large data set of trajectories of buses from the city of Rio de Janeiro. The results confirm the practical applicability of those, since the accomplished performance was as good as that of other state-of-the-art rival methods in most test scenarios.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

The current Big Data epoch is directly related to the trending composition of devices used in daily living activities with a profusion of sensors, targeting improved interaction with their users. Among these sensors, those which gather geographic location information are some of the most popular. That is, a profusion of objects are now shipped with geolocation capability, varying from wearable gadgets to autonomous vehicles. All these devices are then providing data about their real time location by global navigation satellite systems such as the GLONASS, Galileo, GAGAN and the well-known Global Positioning System (GPS).

These generated location data have been used in a plethora of applications and their use has become an important research subject, drawing attention from researchers of various fields: e.g., geography, urban planning, urban mobility, engineering and computer science. The scope of global position data problems is very wide, varying from predictive modelling, pattern mining and discovering to urban service and many others [1]. Our work explored yet another use of geolocation data known as trajectory classification [1,2], wherein the main goal is to be able to label trajectories as elements of a set of classes given extracted spatial features. Such task can be straightforwardly related to the regular classification machine learning task.

Some of the works in trajectory classification include [3], where authors incorporate information about the duration of events in a framework to classify hurricanes (although the framework is tested in different scenarios) using SVM, C4.5 and NaiveBayes approaches. Inferring transportation mode is also common, as in [4–6], using SVM and other mainly used classifiers, and more recently [7] which uses Random Forest and XGBoost techniques. More general frameworks can be found in the literature, such as TraClass [8], that was tested against an animal GPS trajectory database but is claimed to be applicable to any trajectory classification problem. Another closely related work is [9], where authors try to classify bus lines into specific clusters also using artificial neural networks. In addition, we highlight that artificial intelligence and public transit management areas have been connected for many years, with applications ranging from quality assessment to time table prediction, as surveyed by [10].

Our motivation is aimed at an application of GPS data from the bus system of the city of Rio de Janeiro. Like many other public transit systems, Rio de Janeiro has its entire fleet equipped with real-time tracking apparatus. This enables the generation of data records composed of a time stamp, vehicle id, route tag, latitude, longitude and speed for every vehicle running on the streets. A preliminary analysis shows that out of all records, over 200 millions, more than 25% could be discarded because of the lack of a route tag, what is aligned with past reports in the literature [11]. These and other noisy aspects of the data prevent its straightforward use. In this regard, it was previously verified that some vehicles are labelled with one route tag but actually serving another.

* Corresponding author.

E-mail address: d.carvalho@ieee.org (D. Carvalho).

This impairs tasks as to confirm if each route is being served with the officially contracted number of buses. So, in order to verify the fulfilment of Rio's transit regulations, we propose to use a classifier to cross-check the tags of the trajectories, empowering city management with more reliable data.

In this context, we proposed and evaluated three approaches to identify public buses routes from their preexisting digitized GPS trajectories. We performed experimental tests using a lightweight classification architecture known as the WiSARD [12] to help us deal with the data deluge. The WiSARD model has been used in many different previous works and contexts [13–15], always with competitive running time, accuracy and low standard deviation. We also compared WiSARD performance with other mainstream classifiers [7] such as Random Forest and Naive Bayes.

A derived goal of our work is to support traffic operation and optimization in the context of a large city as Rio de Janeiro, wherein 3.7 million of people commute every day by bus. Our contribution aims to in a near future use geolocation data from city's buses to pinpoint deviations from a regular operation. Besides, we believe that our framework can be generalized for other geospatial and non-geospatial classification problems.

The approaches we developed are examples of neuro-symbolic learning [16] methods, which combine neural and symbolic reasoning for optimal performance. We adopted a hybrid strategy, using weightless neural networks combined with a set of pre-defined rules. Another kind of neuro-symbolic learning and reasoning was developed by [17], where neural networks are used as building blocks for a complex reasoning system.

The remainder of this text is organized as follows. In Section 2 we describe Rio de Janeiro's public transit and its geographical tracking system. Next, Section 3 presents the base learning model of our work, the WiSARD neural network model. Section 4 describes our approach to the problem, which relies on different image gridding strategies. Section 5 depicts a set of experiments with real data and presents its results showing how we can benefit from using different pre-processing procedures. At last, Section 6 wraps up providing an overview of our findings, as well as some ideas for future related works.

2. Rio's buses GPS trajectories

Today, many cities around the world rely on automatic vehicle location (AVL) systems for their public transportation management. In the literature, there are some recent works [18–20] dealing with many systems where real-time spatial data of public transit is being provided by some source like an online API or a dashboard and or focusing on city management and operations.

The AVL system in Rio de Janeiro public transit has its current legal structure back to the year of 2010 when a set of regulations imposed the real-time geolocation on the entire fleet. Researchers have then been developing work to analyze and discuss the traffic in Rio de Janeiro. In [21] the authors explain a framework for data acquisition and storage, which is essential to our work (we use a similar approach). Additionally, in [11], authors explore an observatory for this real-time data, with travel time discussion.

According to the local government agency, there were around 705 bus routes circulating on Rio de Janeiro's streets during this period. The transit system itself was provided by 44 companies grouped in five consortia, representing more than 9000 vehicles. Monthly, it results in more than 1,500,500 trips, over 1,000,000 passengers carried and beyond 60,000,000 kilometers traveled.

3. The WiSARD weightless neural network architecture

The WiSARD model we test here has its name derived from its authors (Wilkie, Stonham e Aleksander's Recognition

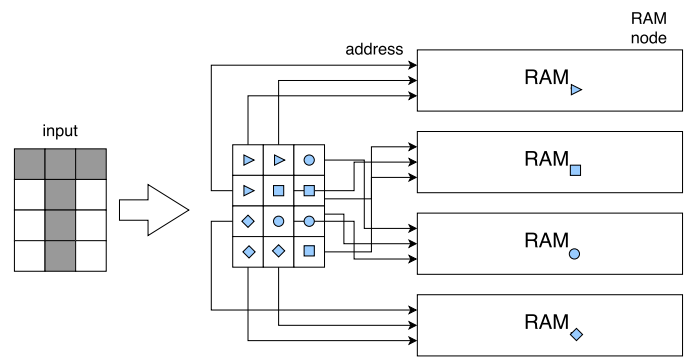


Fig. 1. Training phase using WiSARD: the input binary vectors are pseudorandomly mapped into n -bit tuples ($n = 3$) that become addresses of X RAMs ($X = 4$).

Device) [12] and is a weightless neural network originally developed as a hardware architecture for image recognition [22]. This kind of neural network can be viewed as an implementation of the Bledsoe & Browning's n -tuple classifier [23] and works with basic read and write operations in RAM memories. WiSARD is mainly organized in *discriminators* which are sets of X one-bit word RAMs with n inputs (also called *neurons*), being one discriminator frequently used to determine just one class of many given as input to the classifier.

This model can be categorized as a supervised learning method, although has been expanded to unsupervised learning as well [24]. During the training phase, a number of binary vectors (examples) are shown to the classifier. Each input vector is mapped to a *retina*, which is a division of the input vector in X tuples of n bits, commonly in a pseudo-random manner. Following this strategy, each n -bit tuple is used as an address of a RAM memory of 2^n positions, with the n tuples mapping X memories.

Learning from an example is just as simple as writing a '1' in all memory addresses mapped by a single input. Classification is handled by first mapping an input in the same way training examples were. Afterwards, we present the input to all discriminators. The score r_y for a discriminator y is calculated by counting the number of memories mapped by the classification example whose addresses were previously written by any training example. The discriminator that has the greatest score is chosen as the output class.

One of the possible drawbacks of using the standard WiSARD model may appear when handling very noisy data, when the *saturation* of the RAMs can occur. Saturation happens when many classes have high responses from the discriminators. It is a form of overfitting in which all RAMs have most of its addresses written, thus seeing the same patterns in almost every class. An immediate aid to overcome this problem is the adoption of the *bleaching* [25] technique with the so-called DRASiW. DRASiW [26] is a modification of WiSARD where instead of using one-bit word RAM, q -bit word RAMs are used. Hence, different threshold values can be used to activate a RAM, instead of simply marking whether an address was written or not.

Although very fast and competitive, the WiSARD model is just the simplest model of a whole family of weightless neural networks as presented in [22]. For a visual description of its architecture, the reader can refer to Figs. 1, 2 and also [12,22].

4. A weightless neuro-symbolic classifier

Since the WiSARD model works with a binary pattern as its input unit, a common and almost required step towards its use consists of a pre-processing step (as any other classifier [1]), in this case, a binarization (encoding) step. In spite of being common, binarization is an important matter to achieve a good final

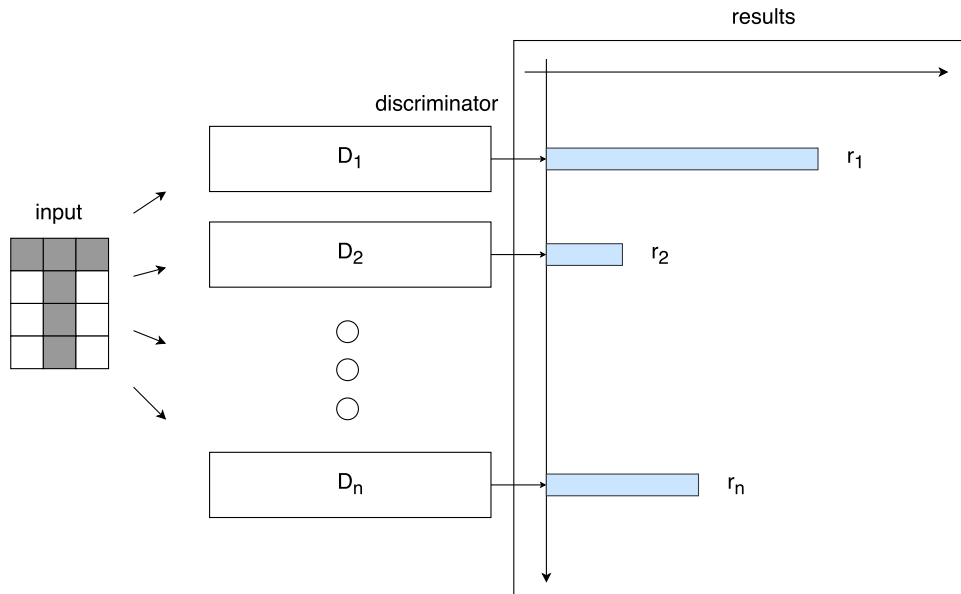


Fig. 2. Classification in the WiSARD: the input (mapped the same way training examples were) is presented to the network and for all trained classes (discriminators), the one with the best result r is chosen.

result when using weightless neural networks [24]. Many authors like [27,28] have also discussed different ways of to convert data into a black/white representation in other scenarios. Binarization has also a common ground with choosing of a good distance metric, which is also related to querying problems [1]. Yet, since our problem is motivated by a specific application, we here propose two main binarization strategies for generating our input and, on top of that, a way of combining classifiers.

4.1. A simple trajectory image

One initial approach to converting our GPS data to a zero-or-one input is to generate a binary image out of the GPS data, composed of a collection of GPS coordinate plotted as points in a 2-D canvas (like in [29]), within some reasonable bounding box such as the city limits. This can be seen as a grid partitioning of the city in disjoint cells similar to [30,31], ignoring the sequential and temporal characteristic of the trajectories. We call this approach *footprint*.

There is a clear trade-off here because it is a lossy compression [32] of the trajectory, but we believe that does impact negatively in the final result. Conversely, one positive aspect that is worth noticing is that, because of this compression, we can tackle a brief or temporary loss of GPS data, as it uses a window of observation and all points in this window are used together as an example. Also, it collapses similar GPS addresses, aiming towards a more complete and maybe redundant trajectory image, avoiding occasional noises.

It is essential to note that due to a WiSARD's characteristic, even if two or more examples have non-overlapping gaps but represent the same class, a complete example can be classified correctly, as the final memory will have a "neural" representation that is equal to the union of all training examples.

Fig. 3 shows a single trajectory – the above-cited footprint – performed by a vehicle that served only one route during a period of time of day. In order to understand the complexity of the classification task, Fig. 4 presents the trajectories of all vehicles, serving all lines during the same time period of time. Hence, the footprint of a vehicle is our unit of classification, representing an example of a bus route for our classifier.

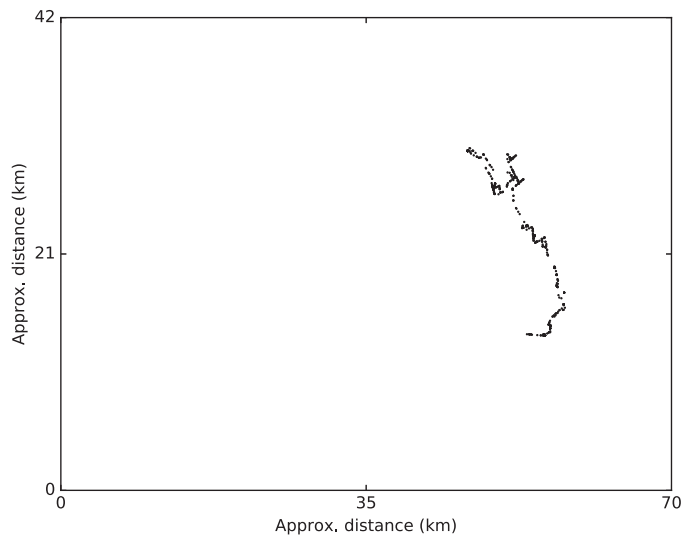


Fig. 3. The footprint of a vehicle.

4.2. A smart gridded image

On top of the previous approach, we also know that we have some prior knowledge about the nature of the data that we are working with. As Fig. 4 shows, taking a small look at the data distribution over the 2-D space can show some regions of interest. This is indeed a visual and statistical proof of something we know beforehand. Rio de Janeiro's relief severely impacts its public transit such that many areas are geographically separated from others, having a single route connecting two very traffic intense areas.

Having that in mind, we then propose to transform our initial grid into a more symbolic representation, readjusting its cells sizes, giving more pixels to regions that might deserve more attention, less to ones that might not. In order to achieve this goal, we re-encode the pixels of the binary image, making a new image out of the leaf nodes of a kd-tree [33] division of our spatial limit. Each pixel of the previous image will certainly fall into a leaf node of this

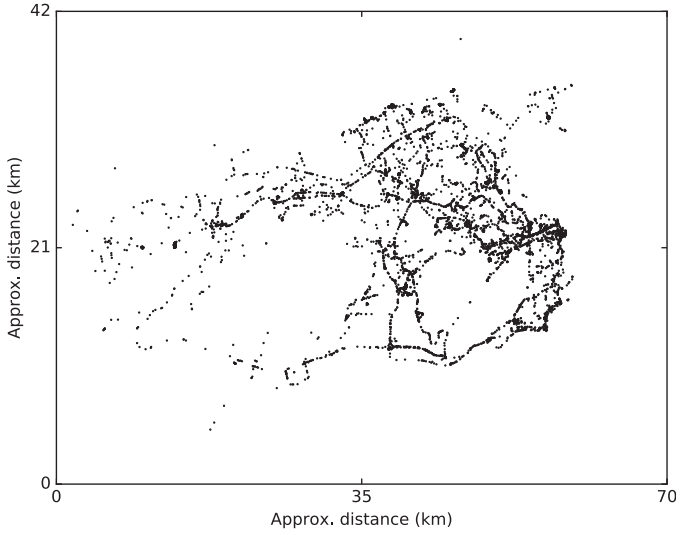


Fig. 4. The footprint of the entire fleet.

kd-tree. Each example is reencoded such that every leaf node with pixels inside gets a '1' in the new image and '0' for the opposite. The input vector is thus transformed to the size of the grid being used (the number of leaves).

This kd-tree is grown using a threshold in the number of leaves and the minimum size of a leaf (a pixel). Also, this division can be guided by different strategies of splitting a leaf: at a median point, for example, and of deciding the next leaf node to be divided (see Algorithm 1 for more details on this procedure). According to

Algorithm 1: The 'kd-tree like' Algorithm.

```

input : nextNodePolicy, splitMethod, nodeLimit
output: tree
while number of nodes < nodeLimit and tree has divisible
nodes do
    nextNodeToSplit ← GetNextNodeToDivide(tree
,nextNodePolicy);
    nextSplittingDimension ←
    GetNextDimensionToDivide(tree);
    SplitBy(tree ,nextNodeToSplit ,nextSplittingDimension
,splitMethod)
end

```

this representation, Fig. 5 shows an example of a grid and one input when using this approach. This approach can be thought as an enhancement of the previous approach, with some sort of region-based clustering (as seen in [8]), or even density-based clustering (as used in [34]) for feature extraction since density can be used when deciding the next node to be divided.

4.3. A DDAG ensemble decision based tree model

Going further on what [35] would define as a neuro-symbolic path, our third proposed approach is an ensemble of WiSARD classifiers. We arrange them as a decision directed acyclic graph - DDAG [36], a strategy for multiclass problems. Using this approach, we try to encode logic rules that hold between the same input represented in different grid sizes.

Suppose that a grid G_1 , built using the aforementioned kd-tree approach, is composed of 4 disjoint cells whose union represent the whole space. Consider also a second grid G_2 , also built using the same strategy with exactly same parameters and data, whose

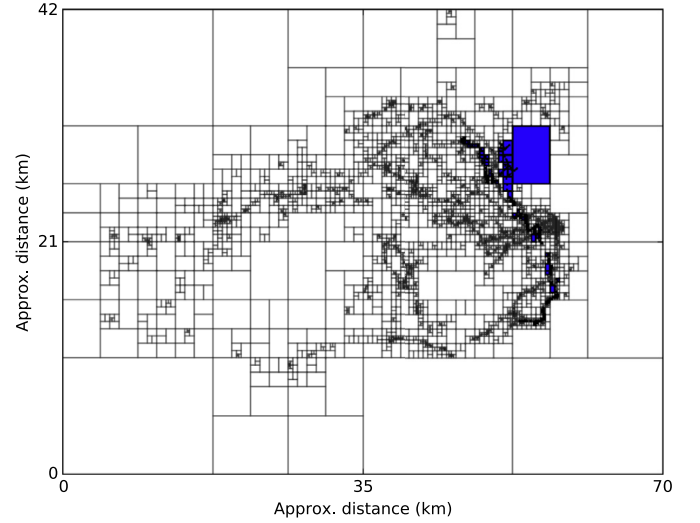


Fig. 5. A footprint example after the grid preprocessing (with grid).

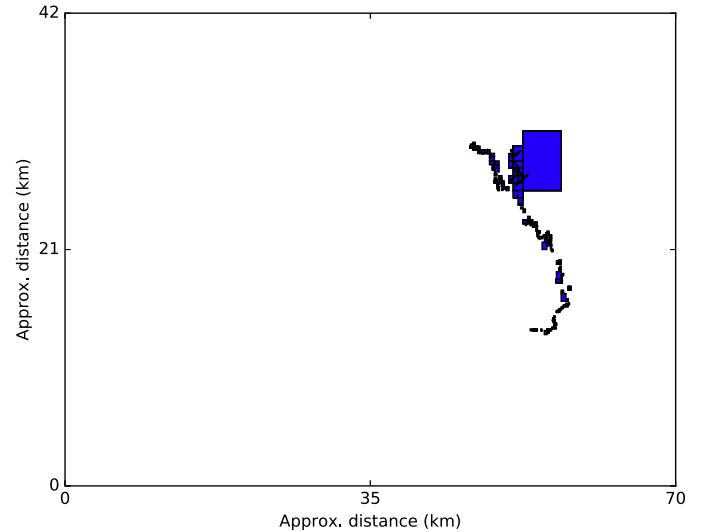


Fig. 6. A footprint example after the grid preprocessing as itself, cell resize occurs in most of cases.

cells union also represent the whole space, but having more cells than G_1 . Then, with luck, each cell in G_1 is the union of 1 or more G_2 cells. If that holds, the following rule applies: a trajectory happening in some subset S_1 of G_1 will happen in some subset S_2 of G_2 whose cells are also contained in S_1 . This idea leads to a tie breaking policy: if two WiSARD discriminators (classes) are in a tie situation using G_1 as input, that tie might not happen when using a more fine-grained input such as G_2 .

Motivated by this idea, this approach work as follows: we train a set of WiSARD classifiers and link them in sequential classification layers. Each layer may conclude or not the classification task. A less detailed WiSARD (small retina) is trained and used to initially classify an example. Following this classifier, different WiSARDs – each one with a more detailed grid – are trained and used to classify the example. The process stops when no class can be identified, only one class remains or the final level is reached, with its output being the model's response. Each layer (or decision node) can be thought of a step of classification using the second approach but with fewer classes to choose from and as if we were applying a zoom to the image. Algorithm 2 tries to put this strategy into picture together with Fig. 7.

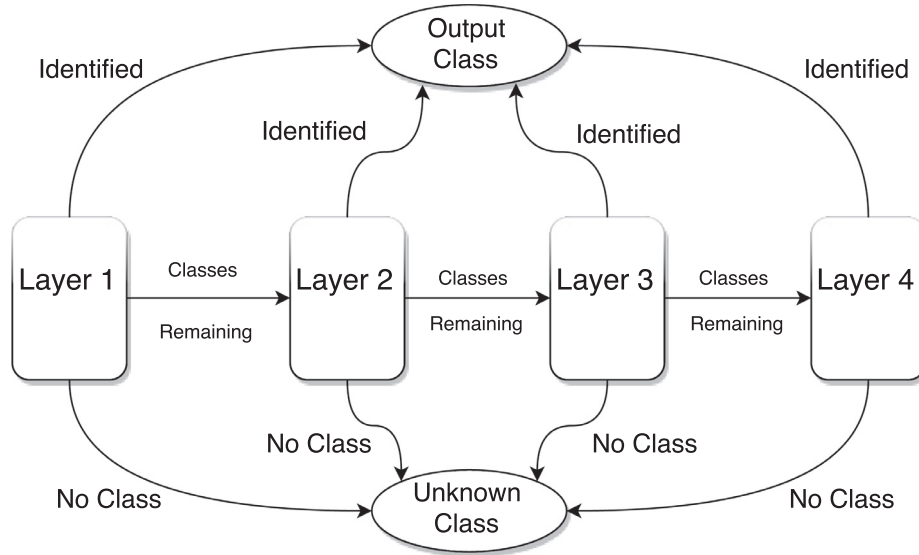


Fig. 7. The decision directed acyclic graph approach with 4 classifier nodes.

Algorithm 2: The Decision DAG Algorithm.

```

input : layers, testSample
output: output
filteredPartialResult ← null;
output ← NOTFOUND;
foreach layer l of pre-trained layers L do
    partialResult ← PredictClass(l, testSample);
    filteredPartialResult ←
        FilterOnlyPreviouslyPredictedClasses(partialResult);
    if filteredPartialResult has only one class then
        if filteredPartialResult[0] == NOTFOUND then
            break;
        else
            output ← filteredPartialResult[0]
        end
    else if filteredPartialResult has more classes then
        output ← random class in filteredPartialResult
    else
        break;
    end
end

```

5. Experimental evaluation

We used a month of data (between June and July of 2015) as our test data in order to evaluate our models. This data was then pre-processed, resulting in images of 620×480 pixels, or input vectors of 297,600 bits. Dataset sizes are up to 50 GB when the original grid is used, with exactly 517 classes to be discriminated.

Regarding the second approach, using the kd-tree (Section 4.2), we tried two splitting methods: dividing a node at the median (*M*) and in half (*H*) – and three heuristics to decide the next node to be divided:

- **Size**: Largest size of node or how many points inside a node (*S*).
- **Density**: Greatest density of a node (*D*)
- **Difference to Mean Density**: Largest absolute difference to the mean density of the current node set (*DMD*).

Also, we tried four different grid/kd-tree sizes (**256**, **1024**, **4096** and **16384** cells/nodes, also the number of bits of each input

vector). In order to evaluate the last model, we decided to take the 4 levels of classifiers (one for each grid size we tested before).

Multiple WiSARD instances were considered, each with a different design: we tested using from 4 to 256 neurons in each discriminator, besides also varying the input grid granularity. The best configuration of WiSARD for each input grid size was also used for DDAG method, in the layer with the same input grid size. Once again, it is important to note that the number of neurons given a fixed-size retina determines also the number of bits used as addresses. In other words, every time *X* neurons are used, if a input vector of *B* bits is used, this leads to the definition of *X* tuples of *n* bits, where $n = B/X$ (assuming that $B \geq X$).

The performed experimental procedure was composed of 5 rounds. In each of these rounds, a random sample of 20 thousand observations was draw from the full data set. Then, a 10-fold cross-validation was performed, so that in each of its (sub)rounds, 18 thousand observations were used as training data, while the remaining 2 thousand examples were used to assess classification effectiveness of the tested alternatives. In this regard, the macro-averaged F1 score [37] was chosen as performance measure due to the imbalanced nature of the data set: some routes have a significantly larger number of records, what could be expected since each route has its own bus frequency.

We also picked three widely used classifiers as baselines for our experiment: Random Forests [38], Extremely Randomized Trees [39] and Naive Bayes [40] (here adapted for a black-and-white input). These methods implementations were obtained from the Scikit Learn [41] package and used with their default configuration. The experiments were realized using an Intel(R) Xeon(R) server, with an E5-2630 2.40 GHz CPU and 32 GB RAM.

Table 1 highlights the top result for each WiSARD approach (out of 181 combinations) as well the baselines compared (Extra Trees, Random Forest and Naive Bayes) ordered by F1 Score, with aggregate training and classification time (excluding initial I/O and pre-processing steps)¹. The best overall result was obtained using the kd-tree approach, using a grid with 16,384 cells and using the policy to split at the half the node with the greatest density. It is interesting to notice that the best model using the DDAG architecture was not able to surpass the original grid strategy, even

¹ The complete table with every experiment can be fetched at <http://bit.ly/2irEpDJ>.

Table 1

Ranking of WiSARD and its derived methods, as well as the baseline alternatives. The reported results regard the best configuration of each method, found by grid-searching through their possible settings. A total of 184 different instances of all classifiers was evaluated.

Solution	Grid Size	# Neurons	F1 score	Avg. Execution time
WiSARD-kd-tree	16,384	64	0.648 $\pm$ 0.005	401.60 $\pm$ 11.28
WiSARD	297,600 (620 $\times$ 480)	128	0.627 $\pm$ 0.004	5916.38 $\pm$ 54.67
WiSARD-DDAG	256/1024/4096/16,384	8/16/64/64	0.622 $\pm$ 0.004	480.40 $\pm$ 1.34
Ext. Rand. Trees	297,600 (620 $\times$ 480)	N/A	0.600 $\pm$ 0.005	1976.55 $\pm$ 67.96
Random forests	297,600 (620 $\times$ 480)	N/A	0.587 $\pm$ 0.005	1381.16 $\pm$ 51.53
Naive Bayes	297,600 (620 $\times$ 480)	N/A	0.027 $\pm$ 0.003	1288.25 $\pm$ 81.89

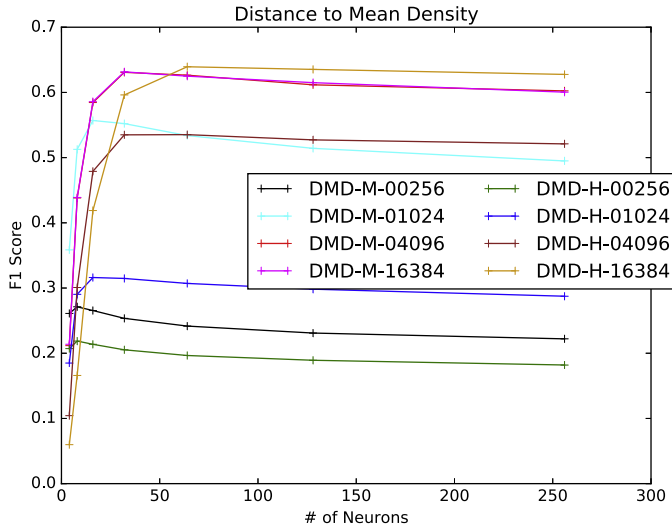


Fig. 8. Results of the WiSARD classifier when using a kd-tree grid with absolute difference to mean density (of points - DMD) of a leaf as criteria for choosing the next node split. Division of node can be either at median (M) or half (H) point, with grid sizes of 256, 1024, 4096 and 16384 cells.

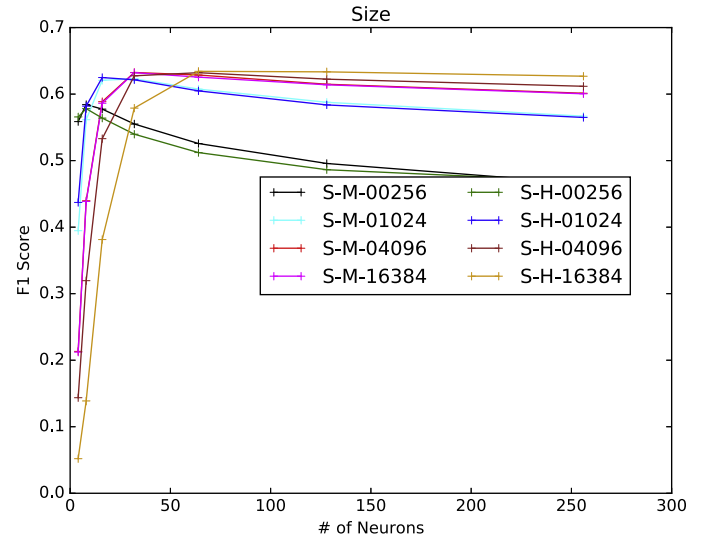


Fig. 9. Results of the WiSARD classifier when using a kd-tree grid with greatest size (S) of leaf as criteria for choosing the next node split. Division of node can be either at median (M) or half (H) point, with grid sizes of 256, 1024, 4096 and 16384 cells.

losing by only 0.02 points. The biggest advantage here is its execution time, being ten times faster, equivalent to the WiSARD-KDTree. This highlights the improvement when using the kd-tree approach over the others, as it is both faster to classify and more accurate.

An aspect denoted by Figs. 8, 9, 12, 10 is the evolution of performance when the number of neurons increases, as all tested architectures seem to reach a peak performance followed by slightly decreases, except from the original grid, where peak performance is reached using 256 neurons. Also, the reader can view a correlation between a grid size (effectively the size of the input) and the number of neurons where the best performance is achieved. Smaller grids tend to reach its peak performance using fewer neurons.

When comparing the performance on F1 score of different classifiers, as depicted by Fig. 11 and 13, we consistently achieve better results when using the WiSARD-KDTree approach. ExtraTrees and Random Forests have similar performance, with the latter reaching a slightly lower performance than the former and with Naive Bayes being definitely not a good option for this situation.

Moreover, it is noticeable the difference of score comparing grid sizes. Both size-driven policies perform relatively similar even using few cells when other strategies tend to show progress when increasing their grid sizes. This might indicate that size-driven grids are more representative of the problem space than other grids.

Concluding, in Fig. 14 we have the results for the DDAG approach, where we see an impressive performance difference depending on the grid strategy, which might be related to the

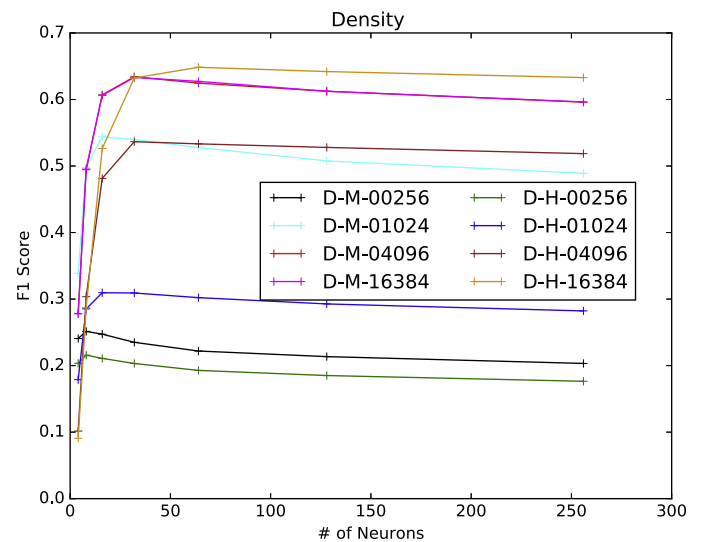


Fig. 10. Results of the WiSARD classifier when using a kd-tree grid with greatest density (D) of a leaf as criteria for choosing the next node split. Division of node can be either at median (M) or half (H) point, with grid sizes of 256, 1024, 4096 and 16,384 cells.

Grids Comparison

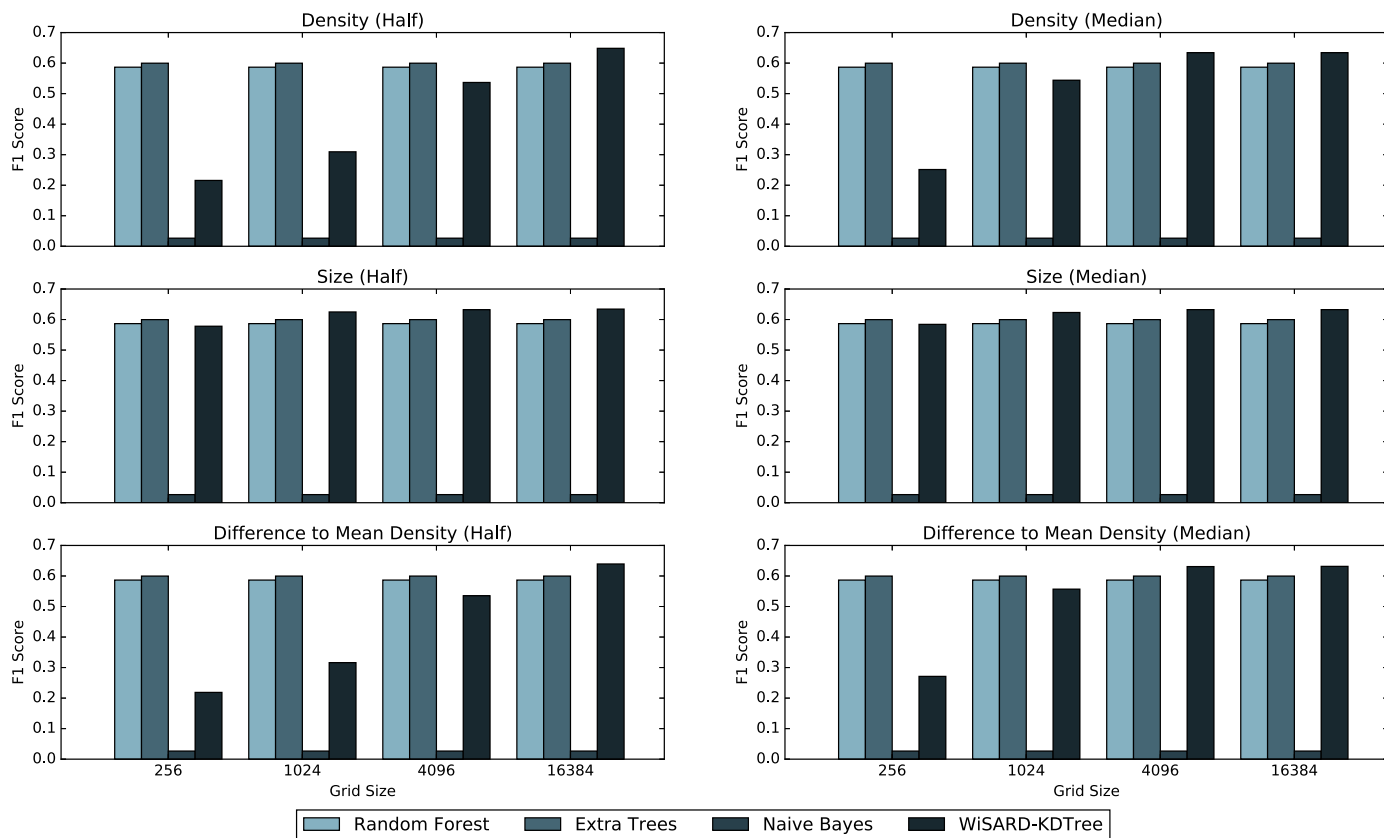


Fig. 11. Comparing Results: WiSARD-KDTree (all grids) vs. Extremely Randomized Trees (ExtraTrees) vs Random Forests vs Naive Bayes.

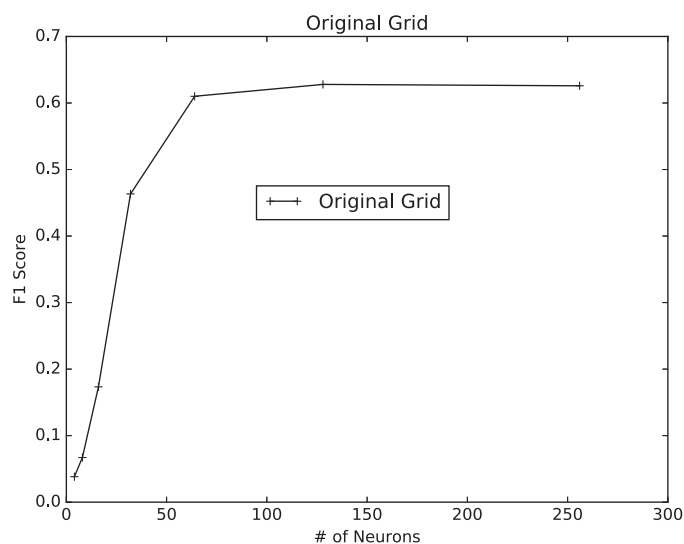


Fig. 12. Results of the WiSARD classifier when using only the original grid strategy.

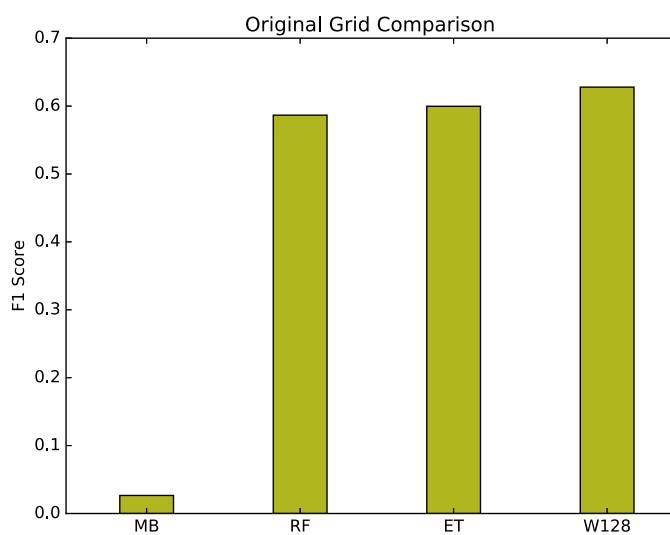


Fig. 13. Comparing Results: Wisard with the original grid using 128 neurons (W128) vs. Extremely Randomized Trees (ET) vs Random Forests (RF) vs Naive Bayes (NB).

representativeness of the grids. It might be the case where having poor grids at a low level ('Density/Half' and 'DMD/Half') severely decreases performance.

6. Conclusion

This paper aims to contribute to the literature of machine learning and knowledge discovery from geographical location data.

More than ever this is an interesting and active research area given the current availability of data of this kind due to the ubiquity of sensors supporting its gathering. Moreover, a very positive influence on urban human life can be achieved based on information resulting from working on this subject. In this regard, our research was based on a very large collection of GPS trajectories from buses in the city of Rio de Janeiro. Identifying labelling inconsistencies

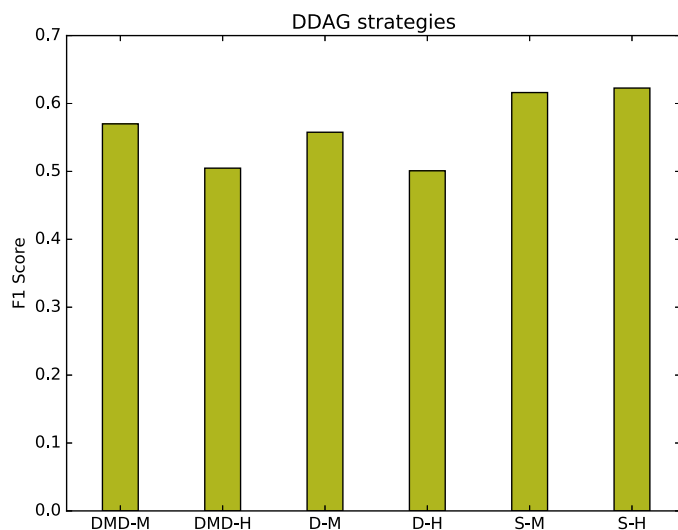


Fig. 14. Results for the WiSARD-DDAG approach. Each result was produced using different kd-tree ensembles, for every possible strategies combination.

through proper classification of such routes could be a valuable asset for traffic optimization and urban engineering.

We developed a neuro-symbolic framework for classification based on GPS data. The WiSARD artificial neural network model is the kernel of such tool, contributing with some of its patent strengths as accuracy and reduced computational cost. We also explored different pre-processing procedures, namely gridding strategies, to transform raw geolocation data into WiSARD-friendly binary inputs. Both WiSARD-based neuro-symbolic learning, as well as binarization techniques compatible with this model, have been approached previously, and this work provides novel findings for both subjects.

The experimental evaluation of the methods proposed showed that its performance is generally at least as good compared to that of other also lightweight, widely used and state-of-the-art classifiers. That is, our WiSARD-based solution has the best overall F1 score, but the results show that it cannot be declared an overall winner since Extra Trees win in some scenarios, when they fed on data pre-processed by our framework. This fact indicates that all classifiers seem to benefit from using a kd-tree quantization instead of a raw image approach.

Since the kd-tree grid led to the best performance, testing other tree structures in this regard or other application scenarios are an interesting follow-up of this work. Also, the presented framework can be an alternative when dealing with large data masses and we believe that it can be used easily adapted to solve other similar problems such as rogue vehicle detection. Moreover, to further investigate the neuro-symbolic, the DDAG-based learning process is another interesting future work, possibly considering the addition of parameters such as a confidence threshold.

Acknowledgments

This work was partially supported by Capes, CNPq, FAPERJ and DIPPG/CEFET-RJ.

References

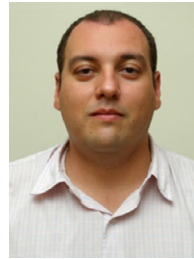
- [1] Y. Zheng, Trajectory data mining: an overview, *ACM Trans. Intell. Syst. Technol.* 6 (3) (2015) 29:1–29:41.
- [2] Z. Feng, Y. Zhu, A survey on trajectory data mining: techniques and applications, *IEEE Access* 4 (2016) 2056–2067.

- [3] D. Patel, C. Sheng, W. Hsu, M.L. Lee, Incorporating duration information for trajectory classification, *Proceedings of the 28th International Conference on Data Engineering (ICDE)*, IEEE, 2012, pp. 1132–1143.
- [4] D.J. Patterson, L. Liao, D. Fox, H. Kautz, Inferring high-level behavior from low-level sensors, *UbiComp*, Springer, 2003, pp. 73–89.
- [5] Y. Zheng, L. Liu, L. Wang, X. Xie, Learning transportation mode from raw GPS data for geographic applications on the web, *Proceedings of the 17th International Conference on World Wide Web*, ACM, 2008, pp. 247–256.
- [6] A. Bolbol, T. Cheng, I. Tsapakis, J. Haworth, Inferring hybrid transportation modes from sparse GPS data using a moving window SVM classification, *Comput. Environ. Urban Syst.* 36 (6) (2012) 526–537.
- [7] Z. Xiao, Y. Wang, K. Fu, F. Wu, Identifying different transportation modes from trajectory data using tree-based ensemble classifiers, *ISPRS Int. J. Geo-Inf.* 6 (2) (2017) 57.
- [8] J.-G. Lee, J. Han, X. Li, H. Gonzalez, Traclust: trajectory classification using hierarchical region-based and trajectory-based clustering, *Proc. VLDB Endow.* 1 (1) (2008) 1081–1094.
- [9] F. Jiménez, F. Serradilla, A. Román, J.E. Naranjo, Bus line classification using neural networks, *Transp. Res. Part D: Transp. Environ.* 30 (2014) 32–37.
- [10] E. Pekel, S. Soner Kara, A comprehensive review for artificial neural network application to public transportation, *Sigma: J. Eng. Nat. Sci./Mühendislik ve Fen Bilimleri Dergisi* 35 (1) (2017).
- [11] K. Rodríguez, M.A. Casanova, L.A.P.P. Leme, H. Lopes, R. Nasser, B.G. Do Amaral, On the design of a traffic observatory application based on bus trajectories, in: *Proceedings of ICEIS* (1), 2016, pp. 215–222.
- [12] I. Aleksander, W. Thomas, P. Bowden, Wisard a radical step forward in image recognition, *Sensor Rev.* 4 (3) (1984) 120–124.
- [13] H.C. Carneiro, C.E. Pedreira, F.M. França, P.M. Lima, A universal multilingual weightless neural network tagger via quantitative linguistics, *Neural Netw.* 91 (2017) 85–101.
- [14] D.O. Cardoso, F.M.G. França, J. Gama, WCDs: A two-phase weightless neural system for data stream clustering, *New Gen. Comput.* 35 (4) (2017) 391–416, doi:10.1007/s00354-017-0018-y, <https://doi.org/10.1007/s00354-017-0018-y>.
- [15] M.D. Gregorio, M. Giordano, Background estimation by weightless neural networks, *Pattern Recogn. Lett.* 96 (Supplement C) (2017) 55–65. Scene Background Modeling and Initialization. <https://doi.org/10.1016/j.patrec.2017.05.029>.
- [16] B. Hammer, P. Hitzler, *Perspectives of Neural-Symbolic Integration*, volume 8, Springer Heidelberg, 2007.
- [17] P.M. Lima, A goal-driven neural propositional interpreter, *Int. J. Neural Syst.* 11 (03) (2001) 311–322.
- [18] G. Padrón, T. Cristóbal, F. Alayón, A. Quesada-Arencibia, C.R. García, System proposal for mass transit service quality control based on GPS data, *Sensors* 17 (6) (2017) 1412.
- [19] C. Tong, H. Chen, Q. Xuan, X. Yang, A framework for bus trajectory extraction and missing data recovery for data sampled from the internet, *Sensors* 17 (2) (2017) 342.
- [20] A. Fiori, A. Mignone, G. Rospo, Decoclu: density consensus clustering approach for public transport data, *Inf. Sci.* 328 (2016) 378–388.
- [21] B.G. Do Amaral, R. Nasser, M.A. Casanova, H. Lopes, Busesinrio: buses as mobile traffic sensors: managing the bus GPS data in the city of Rio de Janeiro, *Proceedings of 17th IEEE International Conference on Mobile Data Management (MDM)*, 2016 volume 1, IEEE, 2016, pp. 369–372.
- [22] F. França, M. De Gregorio, P. Lima, W. de Oliveira, Advances in weightless neural systems, in: *Proceedings of European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, 2014, pp. 497–504.
- [23] W.W. Bledsoe, I. Browning, Pattern recognition and reading by machine, in: *Proceedings of Eastern Joint Computer Conference*, 1959, pp. 225–232.
- [24] D.O. Cardoso, J. Gama, F.M.G. França, Weightless neural networks for open set recognition, *Mach. Learn.* 106 (9–10) (2017) 1547–1567, doi:10.1007/s10994-017-5646-4, <https://doi.org/10.1007/s10994-017-5646-4>.
- [25] B.P. Grieco, P.M. Lima, M. De Gregorio, F.M. França, Producing pattern examples from “mental” images, *Neurocomputing* 73 (7) (2010) 1057–1064.
- [26] C.M. Soares, C.L.F. da Silva, M.D. Gregorio, F.M.G. França, Uma implementação em software do classificador wisard, in: *V Simpósio Brasileiro de Redes Neurais*, 2, 1998, pp. 225–229.
- [27] C. Linneberg, T.M. Jorgensen, Discretization methods for encoding of continuous input variables for boolean neural networks, *Proceedings of International Joint Conference on Neural Networks IJCNN'99*, volume 2, IEEE, 1999, pp. 1219–1224.
- [28] A. Kolcz, N. Allinson, Application of the CMAC input encoding scheme in the n-tuple approximation network, *IEE Proc.-Comput. Digit. Tech.* 141 (3) (1994) 177–183.
- [29] D.F.P. de Souza, Time-series classification with Kernelcanvas and Wisard, Universidade Federal do Rio de Janeiro, 2015 (Master's thesis).
- [30] L. Wang, Y. Zheng, X. Xie, W.-Y. Ma, A flexible spatio-temporal indexing scheme for large-scale GPS track retrieval, *Proceedings of Mobile Data Management*, 2008. MDM'08. 9th International Conference on, IEEE, 2008, pp. 1–8.
- [31] Y. Ge, H. Xiong, Z.-h. Zhou, H. Ozdemir, J. Yu, K.C. Lee, Top-eye: top-k evolving trajectory outlier detection, *Proceedings of the 19th ACM International Conference on Information and Knowledge Management*, ACM, 2010, pp. 1733–1736.
- [32] A. Nibali, Z. He, Trajic: an effective compression system for trajectory data, *IEEE Trans. Knowl. Data Eng.* 27 (11) (2015) 3138–3151.
- [33] J.L. Bentley, Multidimensional binary search trees used for associative searching, *Commun. ACM* 18 (9) (1975) 509–517.

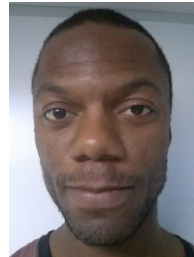
- [34] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al., A density-based algorithm for discovering clusters in large spatial databases with noise., in: Kdd, volume 96, 1996, pp. 226–231.
- [35] G. Pinkas, P. Lima, S. Cohen, Representing, binding, retrieving and unifying relational knowledge using pools of neural binders, *Biol. Inspir. Cogn. Archit.* 6 (2013) 87–95.
- [36] J.C. Platt, N. Cristianini, J. Shawe-Taylor, Large margin DAGs for multiclass classification., in: Proceedings of NIPS, volume 12, 1999, pp. 547–553.
- [37] M. Sokolova, G. Lapalme, A systematic analysis of performance measures for classification tasks, *Inf. Process. Manag.* 45 (4) (2009) 427–437.
- [38] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [39] P. Geurts, D. Ernst, L. Wehenkel, Extremely randomized trees, *Mach. Learn.* 63 (1) (2006) 3–42.
- [40] D. Lewis, Naïve (Bayes) at forty: The independence assumption in information retrieval, in: European conference on machine learning, Springer, Berlin, Heidelberg, 1998, pp. 4–15.
- [41] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: machine learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.



Felipe M. G. França is Professor of Computer Science and Engineering at the Systems Engineering and Computer Science Program, COPPE, Universidade Federal do Rio de Janeiro (UFRJ). He received his B.E.E. (Electronics Engineering, 1981) and his M.Sc. (Computer Science, 1987) from UFRJ, and his Ph.D. (Neural Systems Engineering, 1994) from the Imperial College London. His research and teaching interests include computational intelligence models and all aspects of parallel and distributed computing.



Diego Carvalho is Professor at the Department of Production Engineering (DEPRO) of the Federal Centre for Technological Education of Rio de Janeiro (CEFET/RJ). He got his undergraduate degree in Production Engineering (POLI of UFRJ) and holds a Doctor's Degree in Systems Engineering and Computer Science (PESC/COPPE). Besides, he is the head of the Graduated Programme of Production and Systems Engineering of CEFET/RJ. His research and teaching interests include areas such as object-oriented design, distributed systems, network engineering, parallel architectures, grid technologies, data mining and big data. He is a member of IEEE since 2000 and a member of IEEE Big Data.



Douglas O. Cardoso is an assistant professor at the Department of Computer Engineering of CEFET-RJ Petrópolis, Brazil. He received his B.Sc. in Computer Science, M.Sc. and Ph.D. in Computer and Systems Engineering in 2009, 2012 and 2017, respectively, all from Federal University of Rio de Janeiro (UFRJ). His research interests include clustering, open set recognition, data streams mining and weightless neural networks.



Raul B. Barbosa is a M.Sc. candidate in Computer and Systems Engineering at Federal University of Rio de Janeiro (UFRJ), Brazil. He received his B.Sc. in Computer Science in 2014 also from Federal University of Rio de Janeiro (UFRJ). His research interests include natural language processing, geospatial mining and weightless neural networks.